

Задание

Исходный код

Код после рефакторинга

5.1 Переименование переменной. В классе OrderService метод CalculateFinalPrice использует переменную t. Нужно присвоить более осмысленное название, чтобы значение было понятно другим разработчикам.

```
93 public double CalculateFinalPrice(Order order)
94 {
95     double t = 0.2; // НДС
96     // скидка 10% при заказе от 10000
97     double discount = 0;
98     if (order.Total > 10000)
99     {
100         discount = order.Total * 0.1;
101     }
102     else
103     {
104         discount = 0;
105     }
106     // итоговая цена
107     return order.Total - discount + (order.Total * t);
108 }
109 }
```

5.2 Замена магического числа. В классе OrderService метод CalculateFinalPrice использует «магические числа». Нужно присвоить им более осмысленные названия, чтобы значение было понятно другим разработчикам.

```
93 public double CalculateFinalPrice(Order order)
94 {
95     double t = 0.2; // НДС
96     // скидка 10% при заказе от 10000
97     double discount = 0;
98     if (order.Total > 10000)
99     {
100         discount = order.Total * 0.1;
101     }
102     else
103     {
104         discount = 0;
105     }
106     // итоговая цена
107     return order.Total - discount + (order.Total * t);
108 }
109 }
```

5.3 Упрощение условного выражения. В методе CalculateFinalPrice класса OrderService нужно сделать условное выражение для расчёта скидки более компактным с помощью тернарного оператора.

```
93 public double CalculateFinalPrice(Order order)
94 {
95     double t = 0.2; // НДС
96     // скидка 10% при заказе от 10000
97     double discount = 0;
98     if (order.Total > 10000)
99     {
100         discount = order.Total * 0.1;
101     }
102     else
103     {
104         discount = 0;
105     }
106     // итоговая цена
107     return order.Total - discount + (order.Total * t);
108 }
109 }
```

```
LabWork8
1 using LabWork8.Models;
2
3 namespace LabWork8.Services;
4
5 Ссылка: 3
6 public class OrderPriceCalculator
7 {
8     Ссылка: 1
9     private const double VatRate = 0.20;
10     Ссылка: 1
11     private const double DiscountRate = 0.10;
12     Ссылка: 1
13     private const double DiscountThreshold = 10000;
14
15     Ссылка: 1
16     public double CalculateFinalPrice(Order order)
17     {
18         ArgumentNullException.ThrowIfNull(order);
19
20         double vat = order.Total * VatRate;
21         double discount = order.Total > DiscountThreshold ? order.Total * DiscountRate : 0.0;
22
23         return order.Total - discount + vat;
24     }
25 }
```

```
LabWork8
1 using LabWork8.Models;
2
3 namespace LabWork8.Services;
4
5 Ссылка: 3
6 public class OrderPriceCalculator
7 {
8     Ссылка: 1
9     private const double VatRate = 0.20;
10     Ссылка: 1
11     private const double DiscountRate = 0.10;
12     Ссылка: 1
13     private const double DiscountThreshold = 10000;
14
15     Ссылка: 1
16     public double CalculateFinalPrice(Order order)
17     {
18         ArgumentNullException.ThrowIfNull(order);
19
20         double vat = order.Total * VatRate;
21         double discount = order.Total > DiscountThreshold ? order.Total * DiscountRate : 0.0;
22
23         return order.Total - discount + vat;
24     }
25 }
```

```
Ссылка: 1
1 public double CalculateFinalPrice(Order order)
2 {
3     ArgumentNullException.ThrowIfNull(order);
4
5     double vat = order.Total * VatRate;
6     double discount = order.Total > DiscountThreshold ? order.Total * DiscountRate : 0.0;
7
8     return order.Total - discount + vat;
9 }
10 }
```

5.4 Инкапсуляция поля. В классе Customer поле Name является публичным, что позволяет любому коду изменить его напрямую. Нужно инкапсулировать это поле через свойство и добавить проверку, чтобы значение имени не было пустым или null.

```
25 // Класс Customer (клиент)
26 public class Customer
27 {
28     public int Id { get; set; }
29     public string Name;
30     public string Email;
31     public List<Order> Orders { get; set; }
32 }
```

```
public string Name
{
    get => _name;
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Имя клиента не может быть пустым.", nameof(value));

        _name = value;
    }
}
```

5.5 Перемещение метода. В классе Order имеется ссылка на объект Customer. Метод PrintOrderDetails в OrderService выводит адрес электронной почты клиента. Нужно переместить метод, который выводит информацию о клиенте, в класс Customer, чтобы логика находилась ближе к данным.

```
// Класс Order (заказ)
public class Order
{
    public int Id { get; set; }
    public double Total { get; set; }
    public bool IsExpress { get; set; }
    public int CustomerId { get; set; }
    public Customer Customer { get; set; }
}
```

```
LabWork8
22 }
23
24 Ссылка: 3
25 public string Email { get; set; } = string.Empty;
26 Ссылка: 1
27 public List<Order> Orders { get; set; } = new();
28
29 Ссылка: 1
30 public void PrintDetails()
31 {
32     Console.WriteLine($"Customer: {Name}");
33     Console.WriteLine($"Email: {Email}");
34 }
```

5.6 Извлечение метода. В классе OrderService метод PrintOrderDetails делает несколько вещей: выводит идентификатор заказа, итоговую сумму, статус экспресс-доставки и электронную почту клиента. Нужно разделить эти задачи, выделив каждую часть в отдельные методы.

```
public void PrintOrderDetails(int orderId)
{
    var order = _dbContext.Orders.Include(o => o.Customer).FirstOrDefault(o => o.Id == orderId);
    Console.WriteLine("Order Id: " + order.Id);
    Console.WriteLine("Total: " + order.Total);
    Console.WriteLine("Express Shipping: " + (order.IsExpress ? "Yes" : "No"));
    Console.WriteLine("Customer Email: " + order.Customer.Email);
}
```

```
Ссылка: 1
public void PrintOrderDetails(int orderId)
{
    var order = _dbContext.Orders
        .Include(o => o.Customer)
        .FirstOrDefault(o => o.Id == orderId);

    if (order == null)
    {
        Console.WriteLine($"[Предупреждение] Заказ с ID {orderId} не найден.");
        return;
    }

    PrintOrderIdentifier(order);
    PrintOrderTotal(order);
    PrintExpressShippingStatus(order);
    PrintCustomerContact(order);
}

Ссылка: 1
private void PrintOrderIdentifier(Order order) => Console.WriteLine($"Order Id: {order.Id}");
Ссылка: 1
private void PrintOrderTotal(Order order) => Console.WriteLine($"Total: {order.Total:C}");
Ссылка: 1
private void PrintExpressShippingStatus(Order order) => Console.WriteLine($"Express Shipping: {(order.IsExpress ? "Yes" : "No")}");
Ссылка: 1
private void PrintCustomerContact(Order order)
{
    if (order.Customer != null)
        Console.WriteLine($"Customer Email: {order.Customer.Email}");
}
```

5.7 Замена вложенных условий на guard clauses. В классе CustomerService метод PrintCustomerInfo проверяет customer на null и выводит данные, если он не равен null. Нужно изменить этот код, чтобы использовать guard clauses и сразу выйти из метода, если customer равен null.

```
// Сервис для работы с клиентами
public class CustomerService
{
    private readonly AppDbContext _dbContext;

    public CustomerService(AppDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public void AddCustomer(Customer customer)
    {
        _dbContext.Customers.Add(customer);
        _dbContext.SaveChanges();
    }

    public void PrintCustomerInfo(int customerId)
    {
        var customer = _dbContext.Customers.Include(c => c.Orders).FirstOrDefault(c => c.Id == customerId);
        Console.WriteLine("Customer: " + customer.Name);
        Console.WriteLine("Email: " + customer.Email);
    }
}
```

```
Ссылка: 1
public void PrintCustomerInfo(int customerId)
{
    var customer = _dbContext.Customers
        .Include(c => c.Orders)
        .FirstOrDefault(c => c.Id == customerId);

    if (customer == null)
    {
        Console.WriteLine($"[Предупреждение] Клиент с ID {customerId} не найден.");
        return;
    }

    customer.PrintDetails();
}
```

5.8 Замена длинного метода.
В классе OrderService метод CalculateFinalPrice делает слишком много работы по расчёту итоговой стоимости заказа. Разделить этот метод на несколько частей, выделив в отдельный объект-калькулятор, который будет выполнять расчёт и учитывать различные параметры.

```
public double CalculateFinalPrice(Order order)
{
    double t = 0.2; // НДС
    // скидка 10% при заказе от 10000
    double discount = 0;
    if (order.Total > 10000)
    {
        discount = order.Total * 0.1;
    }
    else
    {
        discount = 0;
    }
    // итоговая цена
    return order.Total - discount + (order.Total * t);
}
```

```
using LabWork8.Models;

namespace LabWork8.Services;

Ссылка: 3
public class OrderPriceCalculator
{
    Ссылка: 1
    private const double VatRate = 0.20;
    Ссылка: 1
    private const double DiscountRate = 0.10;
    Ссылка: 1
    private const double DiscountThreshold = 10000;

    Ссылка: 1
    public double CalculateFinalPrice(Order order)
    {
        ArgumentNullException.ThrowIfNull(order);

        double vat = order.Total * VatRate;
        double discount = order.Total > DiscountThreshold ? order.Total * DiscountRate : 0.0;

        return order.Total - discount + vat;
    }
}
```